

| Tool Type              | Tool Type# |
|------------------------|------------|
| Spot Drill             | 41         |
| Drill                  | 42         |
| Tap                    | 43         |
| End Mil                | 44         |
| Shell Mill             | 45         |
| Ball Nose              | 46         |
| Reserve for Future Use | 47-60      |

## 6.7.4 Variable Usage

All variables are referenced with a number sign (#) followed by a positive number: #1, #10001, and #10501.

Variables are decimal values that are represented as floating point numbers. If a variable has never been used, it can take on a special **undefined** value. This indicates that it has not been used. A variable can be set to **undefined** with the special variable #0. #0 has the value of undefined or 0.0 depending on its context. Indirect references to variables can be accomplished by enclosing the variable number in brackets: # [<Expression>]

The expression is evaluated and the result becomes the variable accessed. For example:

```
#1=3 ;  
#[#1]=3.5 + #1 ;
```

This sets the variable #3 to the value 6.5.

A variable can be used in place of a G-code address where address refers to the letters A-Z.

In the block:

```
N1 G0 X1.0 ;
```

the variables can be set to the following values:

```
#7 = 0 ;  
#1 = 1.0 ;
```

and replaced by:

```
N1 G#7 X#1 ;
```

Values in the variables at runtime are used as the address values.

6.7.5    **Address Substitution**

The usual method of setting control addresses A-Z is the address followed by a number. For example:

```
G01 X1.5 Z3.7 F.02 ;
```

sets addresses G, X, Z, and F to 1, 1.5, 3.7, and 0.02 respectively, and thus instructs the control to move linearly, G01, to position X = 1.5 and Z = 3.7 at a feed rate of 0.02 inches per revolution. Macro syntax allows the address value to be replaced with any variable or expression.

The previous statement can be replaced by this code:

```
#1=1 ;  
#2=0.5 ;  
#3=3.7 ;  
#4=0.02 ;  
G#1 X[#1+#2] Z#3 F#4 ;
```

The permissible syntax on addresses A-Z (exclude N or O) is as follows:

|                            |              |
|----------------------------|--------------|
| <address><variable>        | A#101        |
| <address><-><variable>     | A-#101       |
| <address>[<expression>]    | Z[#5041+3.5] |
| <address><->[<expression>] | Z-[SIN[#1]]  |

If the value of the variable does not agree with the range of the address, then the usual control alarm results. For example, this code results in an invalid G code alarm because there is no G143 code:

```
#1= 143 ;  
G#1 ;
```

When a variable or expression is used in place of an address value, the value is rounded to the least significant digit. If #1=.123456, then G01 X#1 would move the machine tool to .1235 on the X Axis. If the control is in the metric mode, the machine would be moved to .123 on the X axis.

When an undefined variable is used to replace an address value, that address reference is ignored. For example:

```
(#1 is undefined) ;  
G00 X1.0 Z#1 ;
```

becomes

```
G00 X1.0 (no Z movement takes place) ;
```

## Macro Statements

Macro statements are lines of code that allow the programmer to manipulate the control with features similar to any standard programming language. Included are functions, operators, conditional and arithmetic expressions, assignment statements, and control statements.

Functions and operators are used in expressions to modify variables or values. The operators are essential to expressions while functions make the programmer's job easier.

# Functions

Functions are built-in routines that the programmer has available to use. All functions have the form <function\_name>[argument] and return floating-point decimal values. The functions provided in the Haas control are as follows:

| Function | Argument | Returns | Notes                                       |
|----------|----------|---------|---------------------------------------------|
| SIN[ ]   | Degrees  | Decimal | Sine                                        |
| COS[ ]   | Degrees  | Decimal | Cosine                                      |
| TAN[ ]   | Degrees  | Decimal | Tangent                                     |
| ATAN[ ]  | Decimal  | Degrees | Arctangent Same as FANUC ATAN[ ]/[1]        |
| SQRT[ ]  | Decimal  | Decimal | Square root                                 |
| ABS[ ]   | Decimal  | Decimal | Absolute value                              |
| ROUND[ ] | Decimal  | Decimal | Round off a decimal                         |
| FIX[ ]   | Decimal  | Integer | Truncate fraction                           |
| ACOS[ ]  | Decimal  | Degrees | Arc cosine                                  |
| ASIN[ ]  | Decimal  | Degrees | Arcsine                                     |
| #[ ]     | Integer  | Integer | Indirect reference Refer to page <b>224</b> |

## Notes on Functions

The function `ROUND` works differently depending on the context that it is used. When used in arithmetic expressions, any number with a fractional part greater than or equal to .5 is rounded up to the next whole integer; otherwise, the fractional part is truncated from the number.

```
%  
#1=1.714 ;  
#2=ROUND[#1] (#2 is set to 2.0) ;  
#1=3.1416 ;  
#2=ROUND[#1] (#2 is set to 3.0) ;  
%
```

When `ROUND` is used in an address expression, metric and angle dimensions are rounded to three-place precision. For inch dimensions, four-place precision is the default.

```
%  
#1= 1.00333 ;  
G00 X[ #1 + #1 ] ;  
(Table X Axis moves to 2.0067) ;  
G00 X[ ROUND[ #1 ] + ROUND[ #1 ] ] ;  
(Table X Axis moves to 2.0067) ;  
G00 A[ #1 + #1 ] ;  
(Axis rotates to 2.007) ;  
G00 A[ ROUND[ #1 ] + ROUND[ #1 ] ] ;  
(Axis rotates to 2.007) ;  
D[1.67] (Diameter rounded up to 2) ;  
%
```

## Fix vs. Round

```
%  
#1=3.54 ;  
#2=ROUND[#1] ;  
#3=FIX[#1].  
%
```

#2 will be set to 4. #3 will be set to 3.

## Operators

Operators have (3) categories: Boolean, Arithmetic, and Logical.

### Boolean Operators

Boolean operators always evaluate to 1.0 (TRUE) or 0.0 (FALSE). There are six Boolean operators. These operators are not restricted to conditional expressions, but they most often are used in conditional expressions. They are:

EQ - Equal To

NE - Not Equal To

GT - Greater Than

LT - Less Than

GE - Greater Than or Equal To

## LE - Less Than or Equal To

Here are four examples of how Boolean and Logical operators can be used:

| Example                                  | Explanation                                                                                            |
|------------------------------------------|--------------------------------------------------------------------------------------------------------|
| IF [#10001 EQ 0.0] GOTO100 ;             | Jump to block 100 if value in variable #10001 equals 0.0.                                              |
| WHILE [#10101 LT 10] DO1 ;               | While variable #10101 is less than 10 repeat loop DO1..END1.                                           |
| #10001=[1.0 LT 5.0] ;                    | Variable #10001 is set to 1.0 (TRUE).                                                                  |
| IF [#10001 AND #10002 EQ #10003] GOTO1 ; | If variable #10001 AND variable #10002 are equal to the value in #10003 then control jumps to block 1. |

## Arithmetic Operators

Arithmetic operators consist of unary and binary operators. They are:

|     |                      |                                      |
|-----|----------------------|--------------------------------------|
| +   | - Unary plus         | +1.23                                |
| -   | - Unary minus        | -[COS[30]]                           |
| +   | - Binary addition    | #10001=#10001+5                      |
| -   | - Binary subtraction | #10001=#10001-1                      |
| *   | - Multiplication     | #10001=#10002*#10003                 |
| /   | - Division           | #10001=#10002/4                      |
| MOD | - Remainder          | #10001=27 MOD 20 (#10001 contains 7) |

## Logical Operators

Logical operators are operators that work on binary bit values. Macro variables are floating point numbers. When logical operators are used on macro variables, only the integer portion of the floating point number is used. The logical operators are:

OR - logically OR two values together

XOR - Exclusively OR two values together

AND - Logically AND two values together

Examples:

```
%  
#10001=1.0 ;  
#10002=2.0 ;  
#10003=#10001 OR #10002 ;  
%
```

Here the variable #10003 will contain 3.0 after the OR operation.

```
%  
#10001=5.0 ;  
#10002=3.0 ;  
IF [[#10001 GT 3.0] AND [#10002 LT 10]] GOTO1 ;  
%
```

Here control transfers to block 1 because #10001 GT 3.0 evaluates to 1.0 and #10002 LT 10 evaluates to 1.0, thus 1.0 AND 1.0 is 1.0 (TRUE) and the GOTO occurs.



**NOTE:**

*To achieve your desired results, be very careful when you use logical operators.*

## Expressions

Expressions are defined as any sequence of variables and operators surrounded by the square brackets [ and ]. There are two uses for expressions: conditional expressions or arithmetic expressions. Conditional expressions return FALSE (0.0) or TRUE (any non zero) values. Arithmetic expressions use arithmetic operators along with functions to determine a value.

## Arithmetic Expressions

An arithmetic expression is any expression using variables, operators, or functions. An arithmetic expression returns a value. Arithmetic expressions are usually used in assignment statements, but are not restricted to them.

Examples of Arithmetic expressions:

```
%  
#10001=#10045*#10030 ;  
#10001=#10001+1 ;  
X[#10005+COS[#10001]] ;  
#[#10200+#10013]=0 ;  
%
```

## Conditional Expressions

In the Haas control, all expressions set a conditional value. The value is either 0.0 (FALSE) or the value is nonzero (TRUE). The context in which the expression is used determines if the expression is a conditional expression. Conditional expressions are used in the **IF** and **WHILE** statements and in the **M99** command. Conditional expressions can make use of Boolean operators to help evaluate a **TRUE** or **FALSE** condition.

The **M99** conditional construct is unique to the Haas control. Without macros, **M99** in the Haas control has the ability to branch unconditionally to any line in the current subprogram by placing a **P** code on the same line. For example:

```
N50 M99 P10 ;
```

branches to line **N10**. It does not return control to the calling subprogram. With macros enabled, **M99** can be used with a conditional expression to branch conditionally. To branch when variable **#10000** is less than 10 we could code the above line as follows:

```
N50 [#10000 LT 10] M99 P10 ;
```

In this case, the branch occurs only when **#10000** is less than 10, otherwise processing continues with the next program line in sequence. In the above, the conditional **M99** can be replaced with

```
N50 IF [#10000 LT 10] GOTO10 ;
```



# Assignment Statements

Assignment statements let you modify variables. The format of the assignment statement is:

```
<expression>=<expression>
```

The expression on the left of the equal sign must always refer to a macro variable, whether directly or indirectly. This macro initializes a sequence of variables to any value. This example uses both direct and indirect assignments.

```
%
O50001 (INITIALIZE A SEQUENCE OF VARIABLES) ;
N1 IF [#2 NE #0] GOTO2 (B=base variable) ;
#3000=1 (Base variable not given) ;
N2 IF [#19 NE #0] GOTO3 (S=size of array) ;
#3000=2 (Size of array not given) ;
N3 WHILE [#19 GT 0] DO1 ;
#19=#19-1 (Decrement count) ;
#[#2+#19]=#22 (V=value to set array to) ;
END1 ;
M99 ;
%
```

You could use the above macro to initialize three sets of variables as follows:

```
%
G65 P300 B101. S20 (INIT 101..120 TO #0) ;
G65 P300 B501. S5 V1. (INIT 501..505 TO 1.0) ;
G65 P300 B550. S5 V0 (INIT 550..554 TO 0.0) ;
%
```

The decimal point in B101., etc. would be required.

## Control Statements

Control statements allow the programmer to branch, both conditionally and unconditionally. They also provide the ability to iterate a section of code based on a condition.